

بسم الله رحمان رحيم
(داکيومنت SaediOS)

مقدمه:

این صرفا یه کرنل ساده و یه فریمورک ساده برای توسعه نرم افزار در کامپیوتر های (x86) هستش سعی شده با همکاری شما این رو بهترین شکل توسعه بدیم.

علی هادی الساعدی هستم
شماره تماس 09935660353

کانال روبیکا: <https://rubika.ir/SaediOS>

آیدی من در روبیکا: <https://rubika.ir/Aliahadiq>

این نسخه (0.02beta) شامل:

Makefile , kernel.c , color.h , log.h , linker.ld , grub.cfg

*تغییرات جدید:

داکیومنت و تغییر پوشه بندی و (Makefile) رفع یه سری باگ ها اضافه شدن
color.h , log.h

بسم الله رحمان رحيم
(داکيومنت SaediOS)

ويرايست اول مطابق با ورژن (0.02beta)

فهرست

مقدمه صفحه 1

پيش نياز هاى (Makefile) صفحه 3

دستورات (Makefile) صفحه 4 تا 5

نحوه عملکرد بوت لودر در کرنل صفحه 6 تا 9

نحوه عملکرد لينکر صفحه 10 تا 13

نحوه ساخت فايل iso صفحه 13

کتابخانه log.h صفحه 14 تا 17

کتابخانه color.h صفحه 18 تا 19

بسم الله رحمان رحيم (داکيومنت SaediOS)

Makefile:

برای استفاده از (Makefile) باید به اون مسیری که توش (Makefile) وجود داره مراجعه بکنید

برای مثال (Makefile) در این مسیر وجود داره پایین رو بنگر

ali@ali-System-Product-Name:~/Desktop/SaediOS\$

تمامی دستورات (Makefile) در این مسیر قابل اجرا هستن

دوستان برای جلوگیری از ارور ها پیش نیاز ها رو نصب کنید

make install-apt

نصب پیش نیاز برای هایی سیستم هایی که از (apt) استفاده میکنن

make install-pacman

نصب پیش نیاز برای هایی سیستم هایی که از (pacman) استفاده میکنن

-(3)-

```
install-apt:
sudo apt update
sudo apt install -y \
build-essential \
gcc-multilib \
grub-pc-bin \
xorriso \
qemu-system-x86 \
nasm \
binutils\
gcc-multilib\
g++-multilib

install-pacman:
sudo pacman -Sy
sudo pacman -S --needed \
base-devel \
gcc \
grub \
qemu \
qemu-arch-extra \
nasm \
binutils \
xorriso \
gdb \
dosfstools \
mtools\
gcc-multilib\
g++-multilib

#NEW (0.02)
# gcc-multilib g++-multilib
```


بسم الله رحمان رحيم
(داکيومنت SaediOS)

ليست دستورات:

*يه نکته جالب و هم همه کد ها تويه پوشه کد هستن و خروجی اونها توی پوشه (export)
نگه داری همیشه!

make ckernel

کامپیل کرنل و خروجی ابجکت فایل (kernel.o)

make linker

لینک کردن کرنل و خروجی (kernel.bin)

make making-iso

ترکیب با بوت لودر (grub) و ساخت فایل (iso)

make qemu

ران کردن فایل (SaediOS.iso) در شبیه ساز (qemu)

make rm-all

حذف تمامی فایل خروجی جز فایل (SaediOS.iso)

make rm-all-iso

حذف تمامی فایل ها به و به خصوص (SaediOS.iso)

make export-all

خروجی گرفتن از همه فایل ها تو تولید فایل (SaediOS.iso)

بسم الله رحمان رحيم
(داکيومنت SaediOS)

make export-all-rm

خروجی از همه فایل ها و تولید فایل (SaediOS.iso) و حذف فایل های خروجی اضافی

make export-all-qemu

خروجی گرفتن از همه فایل ها تو تولید فایل (SaediOS.iso) و اجرای آن روی (qemu)

make export-all-qemu-rm

خروجی گرفتن از همه فایل ها تو تولید فایل (SaediOS.iso) و اجرای آن روی (qemu) و حذف
فایل های اضافی

بسم الله رحمان رحيم (داکيومنت SaediOS)

نحوه عملکرد (بوتلودر در کرنل)

ابتدا ما به فایل (kernel.c) داریم که هسته سیستم عامل ما هستش خب ببینیم چطوری کار میکنه ؟

ارتباط با بوت لودر

* نکته پیش نیاز این قسمت مطالعه قسمت (Makefile) هستش !

```
#define MULTIBOOT_MAGIC 0x1BADB002
#define MULTIBOOT_PAGE_ALIGN (1 << 0)
#define MULTIBOOT_PAGE_INFO (1 << 0)
#define MULTIBOOT_FLAGS (MULTIBOOT_PAGE_ALIGN | MULTIBOOT_PAGE_INFO)

//Multiboot Header
const struct MultibootHeader {
    unsigned int magic;
    unsigned int flags;
    unsigned int checksum;
} multiboot_header = {

    .magic = MULTIBOOT_MAGIC,
    .flags = MULTIBOOT_FLAGS,
    .checksum = -(MULTIBOOT_MAGIC + MULTIBOOT_FLAGS)
};
```

* نکته (1 << 0) (*همون) (0x10000000) هستش !

بسم الله رحمان رحيم (داکيومنت SaediOS)

تو این قسمت یه عدد جادوی بوت لودر داریم که همون رو تعریف کردیم و یه سری مقادیر رو مقدار دهی کردیم و در نهایت باهم جمعشون زدیم خب بیایم پایین تر

```
#define MULTIBOOT_MAGIC 0x1BADB002 <= عدد جادویی  
#define MULTIBOOT_PAGE_ALIGN (1 << 0) <= مقدار دهی  
#define MULTIBOOT_PAGE_INFO (1 << 0) <= مقدار دهی  
#define MULTIBOOT_FLAGS (MULTIBOOT_PAGE_ALIGN | MULTIBOOT_PAGE_INFO) <= جمع زدن
```

MultibootHeader:

تو این قسمت یه سری متغیر اصلی رو تعریف میکنیم که بالا تر مقدار های ثابتشون رو تعریف کردیم

```
const struct MultibootHeader {  
    unsigned int magic; <= تعریف  
    unsigned int flags; <= تعریف  
    unsigned int checksum; <= تعریف  
}
```

multiboot_header:

حالا مقدار دهیش میکنیم و در نهایت جهت اطمینان تطابق با عدد جادوی بوت لودر (0xBADB002) بررسی میکنیم آیا برابر هست یا نه با متغیر checksum چطوری؟ (صفحه بعد)

```
multiboot_header = {  
    .magic = MULTIBOOT_MAGIC, <= مقدار دهی  
    .flags = MULTIBOOT_FLAGS, <= مقدار دهی  
    .checksum = -(MULTIBOOT_MAGIC + MULTIBOOT_FLAGS) <= بررسی و مقدار دهی  
};
```


بسم الله رحمان رحيم
(داکيومنت SaediOS)

خب ببينيد

MULTIBOOT_MAGIC رو قبل تعريف کرديم که حاويه عدد جادوی بود MULTIBOOT_FLAGS
اين رو هم قبل تعريف کرديم که حاصل جمع دو عبارت مقدار دهی شده بود خب بايد اين دو
رو از هم کم بکنيم بايد مقدار صفر بشه واگر نه بوت لودر ارور ميده

خب حالا ميتونيم کرنل رو به بوت لودر معرفی بکنيم

الان برای اين قسمت کافيه بعد ها بيشتر صحبت ميکنيم

حالا بايد کامپيلش بکنيم با دستور

Make ckerne

خروجی (kernel.o)

اگه ارور داد يه سری به قسمت (makefile) بزنييد کلا خوبه

در اصل اينه

```
gcc -m32 -c kernel.c -o kernel.o -ffreestanding -O2  
-Wall -Wextra
```


بسم الله رحمان رحيم
(داکيومنت SaediOS)

-m32

يعنى خروجى 32 بيت

-ffreestanding

يعنى محيط ازاد از ستاندارد ها همون مناسب براى کرنل

-wall

نمايش هشدار ها هنگام كامپايل

-Wextra

نمايش هشدار هاى بيشتر

-O2

يعنى بدون صرف زمان بيشتر سعى كنه كد ما رو بهينه تر بكنه!

بسم الله رحمان رحيم
(داکيومنت SaediOS)

نحوه عملکرد (لینکر)

اصلا لینکر چیه لینکر مشخص میکنه چه مقدار از رم رو برای متغیر ها قرض بگیریم

خطوط کدما قسمتی از رم رو اشغال میکنن این هم تعریفش $\leq$ (text).

(.data) در اینجا داده های ثابت متغیر ها رو ذخیره میکنیم!

نمونه ها:

const , #define

(.bss) در اینجا داده های متغیر های غیر ثابت (متحرک) رو ذخیره میکنیم!

* نمونه های کلی:

```
int o = 0; // متحرک  
o = 1;  
ok
```

```
#define tset 0 // ثابت  
test = 1;  
error!
```

```
const int o = 0; // ثابت  
o = 1;  
error!
```


بسم الله رحمان رحيم
(داکيومنت SaediOS)

اگه متحرک نباشه از (const , #define) استفاده بکنیم چون
اگه بخواییم تغییرشون بدیم ارور میدن پس اینا رو توی (.bss) ذخیره میکنیم ولی اگه
تغییر پذیر باشن توی (.data) ذخیره میکنیم!

الان بیایم (liker.ld) رو بررسی بکنیم!

```
ENTRY(kernel_main)
```

```
SECTIONS {
```

```
    . = 1M;
```

```
    .text ALIGN(2K){  
        *(.text)  
    }
```

```
    .data ALIGN(2K){  
        *(.data)  
    }
```

```
    .bss ALIGN(2K){  
        *(.bss)  
    }  
}
```

خب (ENTRY) میگه از کجا شروع بکنیم از کدوم تابع؟
که اینجا ما (kernel_main) رو قرار دادیم !

بسم الله رحمان رحيم
(داکيومنت SaediOS)

SECTIONS یعنی بخش ها !

(. = 1m) یعنی از ابتدای محل دسترس بودن رم یک مگابایت رو رزرو بکن !

ALIGN(2K) یعنی دو کیلوبایت به این مجموعه اختصاص بده !

مثال کلی:

```
.bss ALIGN(2K):{  
    *(.bss)  
}
```

از زیر مجموعه یک مگابایت باشه با چی مشخص میکنیم با یه نقطه مثل (.bss)
پس تو مجموعه (.bss) یه (bss) هست !

حالا میتویم کامپایل بکنیم

make liker

در اصل اینه

```
ld -m elf_i386 -nostdlib -T linker.ld -o kernel.bin kernel.o
```

-m elf_i386

یعنی خروجی 32 بیت و از نوع (ELF) باشه !

بسم الله رحمان رحيم
(داکيومنت SaediOS)

-nostdlib

يعنى بدون کتابخانه هاى پويا معمولا کرنل از کتابخانه هاى سفارشی استفاده می کند

-T linker.ld -o kernel.bin kernel.o

میگه از فایل لینکر شخصی سازی شده استفاده کن و از کرنل دات او استفاده کن و اسمش باشه (kernel.bin)

تولید فایل (iso):

اول ساخت یه سه پوشه درون هم (iso/boot/grub)

mkdir -p iso/boot/grub

دوم جابه جای فایل (grub.cfg) به این مسیر (iso/boot/grub)

cp grub.cfg iso/boot/grub

سوم جابه جایی (kernel.bin) به (iso/boot/)

cp kernel.bin iso/boot/

چهارم از (grub-mkrescue) ایجاد یک فایل (SaediOS.iso)

grub-mkrescue -o \$SaediOS.iso iso

rm -r iso // پاک کردن پوشه اضافی

بسم الله رحمان رحيم (داکيومنت SaediOS)

کتابخانه (log.h)

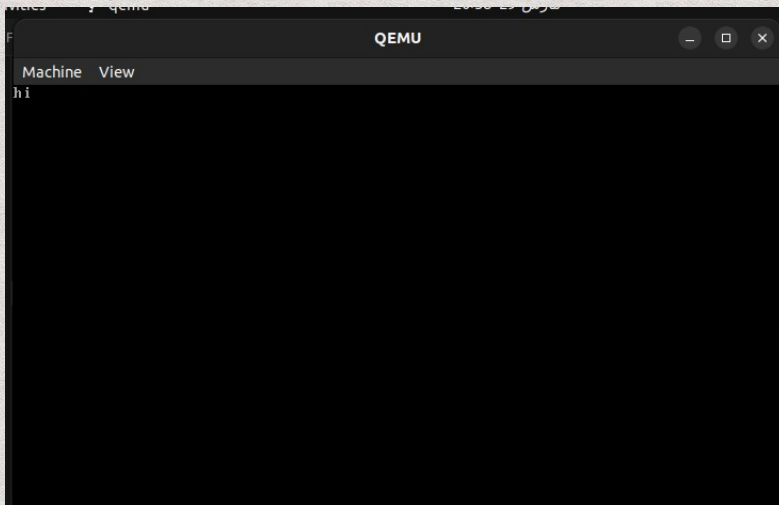
همين طور که از اسم اين کتابخانه هم پيداست اين کتابخانه برای چاپ پيام ها است برای اينکه بتونيم از اين کتابخانه استفاده بکنيم بايد اون رو صدا بزنيم

```
#include "log.h"
```

حالا بايد نحوه کار کرد رو با هم درک بکنيم

```
log("hi",0,0x0,0x7);
```

خروجی این یک (hi) ساده هستش



خب بزاريد دقيق تر بگم اين اعداد و ارقام چی هستن

;(رنگ متن,colorText , رنگ پس زمینه colorBackGround , محل شروع map , "پيام",log);

گفتيم از مختصات صفر شروع بکنه رنگ پس زمینه سياه و رنگ متن خاکستری

0x0 Black , 0x7 gruy

بسم الله رحمان رحيم
(داکيومنت SaediOS)

* نترسيد جلو تر با کتابخانه (color.h) آشنا ميشيم که کار رو راحت تر ميکنه
انواع رنگ ها به هگزاد دسميال:

0x0 = سیاه
0x1 = آبی
0x2 = سبز
0x3 = فیروزهای
0x4 = قرمز
0x5 = بنفش
0x6 = نارنجی
0x7 = سفید
0x8 = خاکستری
0x9 = بنفش
0xA = سبز روشن
0xB = آبی روشن
0xC = قرمز روشن
0xD = بنفش روشن
0xE = زرد روشن
0xF = سفید روشن

تست کنید انواع رنگ ها رو خروجی جالبی رو مشاهده میکنید و اما تو صفحه بعد در مورد
مپ ها صحبت میکنیم

بسم الله رحمان رحيم (داکيومنت SaediOS)

برای رفع این مشکل باید از این کد استفاده بکنید

```
char* text1 = "good";  
char* text2 = "hi";  
  
log (text1,0,0x0,0x7);  
log(text2,sizein(text1)+1,0x0,0x7);
```



خب الان درست شد ولی شاید یه سوال براتون پیش بیاد (sizein()) چیه ؟
این برای بدست آوردن تعداد متنی که تو یه (char*) نوشته شده کاربرد داره که بالا ازش
استفاده کردیم حاصلش میشه 4 که جمعش زدیم با 1 که نتیجهش شد 5 !

یه راه دیگه هم داره اون بامزه تره

```
int map = log ("good",0,0x0,0x7);  
log ("hi",map+1,0x0,0x7);
```

*همیشه (log ("good",0,0x0,0x7);) اندازه مپ اشغال شده رو برای ما بر میگرونه

حالا شما کامل کتابخانه (log.h) رو یاد گرفتید !

بسم الله رحمان رحيم
(داکيومنت SaediOS)

کتابخانه (color.h)

بلا نسبت این کتابخانه بامزه و راحت که میتوانید به جای اعداد هگزاد دسیمال از ارزش استفاده بکنید

نحوه صدا زدن :

`#include "color.h"`

list color:

Red () قرمز
Green () سبز
Yellow () زرد
Blue () آبی
Black () سیاه
White () سفید
ARed () قرمز پررنگ
AGreen () سبز پررنگ
ABlue () آبی پررنگ
AYellow () نارنجی
AWhite () سفید روشن

برای مثال:

`log("test color.h",0,AWhite(),Blue());`

بسم الله رحمان رحيم (داکيومنت SaediOS)

اين هم عکس:



يه نکته بامزه :

```
color text = Green();  
log("tset",0,Black(),text);
```

